Transaction List — Performance Design & Architecture

Version: 1.0
Date: April 15, 2026
Author: Architecture Team
Parent Doc: banking-application-architecture.md

Table of Contents

1. Problem Statement — Why This Is Hard
2. Data Scale & Growth Model
3. End-to-End Request Lifecycle — Transaction List Load
4. Pagination Strategy — Offset vs Cursor and When to Use Which
5. Caching — Every Layer, What and Why
6. Database Partitioning — Handling Billions of Rows
7. Indexing Strategy — Making Queries Fast
8. Query Execution Plan Analysis
9. UI Performance — Loading States, Skeletons & Perceived Speed
10. When Data Takes Too Long — Timeout UX Patterns
11. When It's Consistently Slow — Root Cause Diagnosis & Fixes
12. Read Replica & Connection Pool Strategy
13. Materialized Views & Pre-Computed Summaries
14. Performance Budget — Latency Breakdown Per Layer
15. Load Testing & Performance Validation
16. Performance Monitoring & Alerting

---

1. Problem Statement — Why This Is Hard

The transaction list page is the most visited screen in a banking app. It is also the single hardest screen to keep fast because of the data characteristics:

```
┌──────────────────────────────────────────────────────────────────────┐
│                  WHY TRANSACTION LIST IS HARD                        │
│                                                                      │
│  1. DATA VOLUME                                                      │
│     • 10M transactions/day → 300M/month → 3.6B/year                  │
│     • Each row: ~500 bytes → 1.8 TB/year raw data                    │
│     • With indexes: ~5 TB/year total storage                         │
│                                                                      │
│  2. QUERY PATTERNS ARE ADVERSARIAL                                   │
│     • "Show me all UPI transactions in the last 90 days" → scans     │
│       3 monthly partitions, filtered by category                     │
│     • "Show me transactions > ₹1 lakh last year" → scans 12 partitions │
│     • "Search for Swiggy" → full-text search across 300M+ rows       │
│                                                                      │
│  3. USERS EXPECT INSTANT RESPONSE                                    │
│     • Users tap a filter and expect results in < 1 second            │
│     • Any delay > 2 seconds → user assumes "the app is broken"       │
│     • Mobile users on 3G expect smooth scrolling (infinite scroll)   │
│                                                                      │
│  4. DATA IS LIVE                                                     │
│     • New transactions arrive every second                           │
│     • A transfer completed 5 seconds ago must appear in the list     │
│     • Balances change during the session                             │
│                                                                      │
│  5. ACCESS PATTERNS VARY WILDLY                                      │
│     • 80% of views: "last 30 days, my savings account" (simple)      │
│     • 15% of views: filtered by category + status + amount (moderate) │
│     • 5% of views: search + filters + 90-day range (expensive)       │
│                                                                      │
│  The architecture must make the 80% case blazing fast (< 100ms)      │
│  while keeping the 5% case acceptable (< 500ms).                     │
└──────────────────────────────────────────────────────────────────────┘
```

---

2. Data Scale & Growth Model

2.1 Transaction Table Growth

Timeframe	Rows	Raw Data	With Indexes	Partitions
1 month	300M	150 GB	~400 GB	1 partition
6 months	1.8B	900 GB	~2.4 TB	6 partitions
1 year	3.6B	1.8 TB	~5 TB	12 partitions
3 years	10.8B	5.4 TB	~15 TB	36 partitions

2.2 Per-User Data Profile

User Type	Txns/Month	Txns/Year	Typical Query Range
Low-activity (savings)	20-50	250-600	Last 30 days
Average consumer	100-300	1,200-3,600	Last 30-90 days
High-activity (business)	1,000-5,000	12,000-60,000	Last 7-30 days
Merchant account	10,000-100,000	120,000-1.2M	Last 1-7 days

The per-user data volume is manageable (even a high-activity user has < 100K rows/year). The challenge is querying efficiently across 3.6B total rows to find one user's 3,600 rows in the right time window, category, and status.

---

3. End-to-End Request Lifecycle — Transaction List Load

3.1 Happy Path (Cache Hit — 80% of Requests)

```
User opens "Transaction History" tab
      │
      ▼ (0ms)
[1]  React checks RTK Query cache
     Cache key: ['transactions', { accountId: 'ACC-001', dateRange: '30d', page: 0 }]
     │
┌────┴────┐
│ HIT?    │
└────┬────┘
     │ YES (staleTime=30s not expired)
     ▼ (0ms — instant)
[2]  Render cached data immediately
     Show transaction list with previous results
     ─── DONE ─── Total: 0ms (instant from memory)

     Background: if staleTime expired but cacheTime hasn't:
     → Show stale data immediately, refetch in background
     → When fresh data arrives, seamlessly update the list
     → User sees no loading spinner (stale-while-revalidate)
```

3.2 Cache Miss Path (Full Fetch — 20% of Requests)

```
User opens "Transaction History" (first load or cache expired)
      │
      ▼ (0ms)
[1]  React: No cache hit → show Skeleton UI immediately
     11 skeleton rows with shimmering animation
     Filter panel enabled (user can start selecting filters)
      │
      ▼ (0ms)
[2]  RTK Query dispatches: GET /api/v1/transactions?accountIds=ACC-001&from= … &page=0&size=20
     Debounced 300ms (if triggered by filter change)
      │
      ▼ (~5ms)
[3]  CDN / WAF passthrough (no caching for authenticated API calls)
      │
      ▼ (~10ms)
[4]  API Gateway:
     ├── JWT validation (Redis lookup: session:{hash} — 0.3ms)
     ├── Rate limit check (Redis INCR — 0.2ms)
     └── Route to BFF-Web
      │
      ▼ (~5ms)
[5]  BFF-Web:
     ├── Validate & sanitize filter params
     ├── Extract userId from JWT (server-side, not from client)
     └── Forward to Transaction Service (gRPC)
      │
      ▼ (~5ms)
[6]  Transaction Service:
     ├── Build JPA Specification from filters
     ├── Route decision: text search → ES, filters only → PG
     │
     │ [FILTERS ONLY — PostgreSQL path]
     ├── @Transactional(readOnly = true) → routes to READ REPLICA
     ├── Execute query:
     │     SELECT * FROM transactions
     │     WHERE account_id = 'ACC-001'
     │       AND created_at ≥ '2026-03-16' AND created_at ≤ '2026-04-15'
     │     ORDER BY created_at DESC
     │     LIMIT 20 OFFSET 0
     │
     │   PostgreSQL query planner:
     │   ├── Partition pruning: scan only transactions_2026_03, transactions_2026_04
     │   ├── Index scan: idx_txn_account_date (account_id, created_at DESC)
     │   └── Execution time: ~8ms
     │
     ├── Count query (for pagination total):
     │     SELECT COUNT(*) FROM transactions WHERE … (same filters)
     │     Execution time: ~5ms (or estimated if > 10K)
     │
     └── Map entities to DTOs
      │
      ▼ (~5ms)
[7]  Response flows back: Txn Service → BFF-Web → API Gateway → Client
      │
      ▼ (~0ms)
[8]  React:
     ├── RTK Query stores response in memory cache
```

```
        ├── Replace skeleton with actual transaction rows
        ├── Animate transition (fade-in, 150ms)
        ├── Show pagination: "Page 1 of 12 (234 transactions)"
        └── Show result count: "Showing 20 of 234 transactions"

       ─── DONE ─── Total: ~80-120ms (p95)
```

3.3 Latency Waterfall

```
┌──────────────────────────────────────────────────────────────────┐
│  LATENCY WATERFALL — Transaction List (p95)                       │
│                                                                   │
│  Component               | Time    | Cumulative | % of Total      |
│  ────────────────────────┼─────────┼────────────┼────────────────── │
│  React (dispatch)        |   1ms   |     1ms    |   1%            |
│  Network (client → GW)   |  15ms   |    16ms    |  13%            |
│  API Gateway (auth+RL)   |   5ms   |    21ms    |   4%            |
│  Network (GW → BFF)      |   2ms   |    23ms    |   2%            |
│  BFF (validate+route)    |   3ms   |    26ms    |   3%            |
│  Network (BFF → Svc)     |   2ms   |    28ms    |   2%            |
│  Txn Service (build spec)|  2ms    |    30ms    |   2%            |
│  PG Query Execution      |  15ms   |    45ms    |  13%    ← MAIN     |
│  PG Count Query          |   8ms   |    53ms    |   7%            |
│  DTO Mapping             |   3ms   |    56ms    |   3%            |
│  Network (Svc → BFF)     |   2ms   |    58ms    |   2%            |
│  BFF (response assembly)|   2ms   |    60ms    |   2%            |
│  Network (BFF → GW)      |   2ms   |    62ms    |   2%            |
│  Network (GW → client)   |  15ms   |    77ms    |  13%            |
│  React (render)          |   8ms   |    85ms    |   7%            |
│  ────────────────────────┼─────────┼────────────┼────────────────── │
│  TOTAL                   |  85ms   |    85ms    | 100%            |
│                                                                   │
│  Bottleneck: Network round-trip (28ms) + PG queries (23ms)        │
└──────────────────────────────────────────────────────────────────┘
```

4. Pagination Strategy — Offset vs Cursor and When to Use Which

4.1 The Two Strategies

```
┌──────────────────────────────────────────────────────────────────────┐
│              OFFSET-BASED PAGINATION                                 │
│                                                                      │
│  Request:  GET /transactions?page=3&size=20                          │
│  SQL:      ... ORDER BY created_at DESC LIMIT 20 OFFSET 40           │
│                                                                      │
│  Page 1: OFFSET 0    [row 1-20]    ✓ Fast (index scan, stop at 20)  │
│  Page 2: OFFSET 20   [row 21-40]   ✓ Fast                           │
│  Page 3: OFFSET 40   [row 41-60]   ✓ Fast                           │
│  Page 10: OFFSET 180 [row 181-200] ○ Acceptable                     │
│  Page 50: OFFSET 980 [row 981-1000] △ Slow (scans and discards 980 rows)│
│  Page 500: OFFSET 9980              ✗ Very slow (scans 9980 rows)   │
│                                                                      │
│  Problem: PostgreSQL must scan and discard all rows before OFFSET.   │
│  At OFFSET 9980 with 3.6B total rows, PG still only scans within     │
│  the matching partition + index, but physically skips 9980 index entries. │
│                                                                      │
│  UI: Classic page numbers [1] [2] [3] ... [12] [Next →]              │
│       User can jump to any page. Can see total pages.                │
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────┐
│              CURSOR-BASED PAGINATION (Keyset)                        │
│                                                                      │
│  Request:  GET /transactions?after=2026-04-14T18:30:00Z&size=20      │
│  SQL:      ... WHERE created_at < '2026-04-14T18:30:00Z'            │
│            ORDER BY created_at DESC LIMIT 20                         │
│                                                                      │
│  Page 1: No cursor     [newest 20]     ✓ Fast (index scan, stop at 20)  │
│  Page 2: after=lastRow [next 20]       ✓ Fast (index seek + 20 rows)    │
│  Page 50: after=lastRow [next 20]      ✓ Still fast (same cost as page 1)│
│  Page 500: after=lastRow [next 20]     ✓ Still fast (no OFFSET penalty) │
│                                                                      │
│  The cursor is the created_at + id of the last row on the current page.  │
│  PG seeks directly to that position in the index — no rows skipped.  │
│                                                                      │
│  Problem: Cannot jump to arbitrary page number.                      │
│  User can only go "next" or "previous" (or "load more").             │
│                                                                      │
│  UI: Infinite scroll or [← Previous] [Next →] buttons.               │
│       No page numbers. No "jump to page 7".                          │
└──────────────────────────────────────────────────────────────────────┘
```

4.2 Decision Matrix

Client	Pagination Type	Why
BFF-Web (desktop)	**Offset-based** (with deep-page guard)	Desktop users expect page numbers, "Page 3 of 12". They can jump to a specific page. Most users stay in pages 1-5. Deep pagination (beyond page 500 / offset 10,000) is blocked with a message to narrow filters.
BFF-Mobile (app)	**Cursor-based** (infinite scroll)	Mobile UX convention is infinite scroll. User scrolls down, new transactions load seamlessly. No page numbers. Cursor-based is efficient for this pattern — every "load more" is equally fast regardless of how far the user has scrolled.
Export API	**Cursor-based** (streaming)	Exporting 10K+ transactions to CSV. Cursor-based allows streaming rows without holding all in memory. Each batch fetches the next 1,000 rows from the cursor position.
Admin search	**Offset-based**	Admin users search across all users' transactions. They need page numbers to navigate results and reference specific pages in support tickets ("see page 4, row 3").

4.3 Offset-Based Implementation (Web)

```java
// Controller
@GetMapping("/api/v1/transactions")
public ResponseEntity<PagedResponse<TransactionDTO>> list(
    @Valid TransactionFilterDTO filters,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(defaultValue = "createdAt,desc") String sort
) {
    if (page * size > 10_000) {
        throw new BadRequestException(
            "Cannot paginate beyond 10,000 results. " +
            "Narrow your date range or use the export endpoint."
        );
    }

    Pageable pageable = PageRequest.of(page, Math.min(size, 100), parseSort(sort));
    Page<TransactionDTO> result = transactionService.list(filters, pageable, userId);

    return ResponseEntity.ok(PagedResponse.<TransactionDTO>builder()
        .results(result.getContent())
        .page(result.getNumber())
        .size(result.getSize())
        .totalElements(result.getTotalElements())
        .totalPages(result.getTotalPages())
        .build());
}
```

4.4 Cursor-Based Implementation (Mobile)

```java
// Controller
@GetMapping("/api/v1/transactions/stream")
public ResponseEntity<CursorResponse<TransactionDTO>> listCursor(
    @Valid TransactionFilterDTO filters,
    @RequestParam(required = false) Instant after,       // cursor: created_at of last seen row
    @RequestParam(required = false) UUID afterId,        // tiebreaker: id of last seen row
    @RequestParam(defaultValue = "20") int size
) {
    List<TransactionDTO> results = transactionService.listAfterCursor(
        filters, after, afterId, Math.min(size, 50), userId
    );

    Instant nextCursor = results.isEmpty() ? null : results.getLast().getCreatedAt();
    UUID nextCursorId = results.isEmpty() ? null : results.getLast().getId();

    return ResponseEntity.ok(CursorResponse.<TransactionDTO>builder()
        .results(results)
        .nextCursor(nextCursor ≠ null ? nextCursor.toString() : null)
        .nextCursorId(nextCursorId ≠ null ? nextCursorId.toString() : null)
        .hasMore(results.size() == size)
        .build());
}

// Repository query
@Query("""
    SELECT t FROM Transaction t
    WHERE t.accountId IN :accountIds
      AND t.createdAt ≥ :from AND t.createdAt ≤ :to
      AND (:after IS NULL OR t.createdAt < :after
           OR (t.createdAt = :after AND t.id < :afterId))
    ORDER BY t.createdAt DESC, t.id DESC
    """)
List<Transaction> findAfterCursor(
    List<String> accountIds, Instant from, Instant to,
    Instant after, UUID afterId, Pageable pageable
);
```

Why `created_at + id` as cursor (not just `created_at`)?

Two transactions can have the same `created_at` timestamp (same millisecond). Using `id` as a tiebreaker ensures no rows are skipped or duplicated when paginating through ties.

4.5 Cursor Encoding for the API

The raw cursor values (`after=2026-04-14T18:30:00.123Z&afterId=550e8400-...`) are verbose. We encode them as an opaque Base64 token:

```java
// Encode
String cursor = Base64.encode(nextCursor.toString() + "|" + nextCursorId.toString());
// → "MjAyNi0wNC0xNFQxODozMDowMC4xMjNafDU1MGU4NDAwLS4uLg=="

// Decode
String[] parts = Base64.decode(cursor).split("\\|");
Instant after = Instant.parse(parts[0]);
UUID afterId = UUID.fromString(parts[1]);
```

The client treats the cursor as an opaque string and passes it back on the next request. If the cursor format changes (e.g., adding a new tiebreaker field), the client doesn't need to update.

4.6 Hybrid: Web Gets Both

On desktop, the UI shows page numbers by default. But if the user scrolls to page 50+ or if the result set is very large, the UI can offer to switch to "streaming mode" (infinite scroll with cursor) for better performance:

```
┌──────────────────────────────────────────────────────────┐
│  Page 1 of 500+                                          │
│                                                          │
│  This result set is very large. For better performance,  │
│  switch to streaming mode:                               │
│                                                          │
│  [Continue with page numbers]  [Switch to scroll mode]   │
└──────────────────────────────────────────────────────────┘
```

---

5. Caching — Every Layer, What and Why

5.1 Cache Architecture for Transaction List

```
┌──────────────────────────────────────────────────────────────────────────┐
│                CACHING LAYERS FOR TRANSACTION LIST                        │
│                                                                          │
│  LAYER 1: FRONTEND (React — RTK Query in-memory)                         │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │  Cache key: ['transactions', serializedFilters]                  │    │
│  │  staleTime: 30 seconds                                           │    │
│  │  cacheTime: 5 minutes (GC unused cache entries after 5 min)      │    │
│  │                                                                  │    │
│  │  BEHAVIOR:                                                       │    │
│  │  • t=0: User loads page → cache MISS → fetch from server         │    │
│  │  • t=10s: User navigates away and back → cache HIT (still fresh) │    │
│  │  • t=35s: User comes back → STALE → show cached, refetch in bg   │    │
│  │  • t=6min: Cache entry GC'd → next visit is a full fetch         │    │
│  │                                                                  │    │
│  │  ON FILTER CHANGE: new cache key → always fetch (new combination) │    │
│  │  ON PAGE CHANGE: each page is a separate cache key               │    │
│  │  ON TRANSFER COMPLETE: invalidate all 'transactions' tags        │    │
│  │                        → refetch current view                    │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                                                                          │
│  LAYER 2: BFF (No cache — pass-through)                                  │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │  Transaction lists are NOT cached at BFF.                        │    │
│  │                                                                  │    │
│  │  WHY NOT: Results are personalized (per-user), filter-dependent, │    │
│  │  and paginated. The combinatorial key space (user × filters ×    │    │
│  │  page × sort) makes BFF-level caching infeasible.                │    │
│  │                                                                  │    │
│  │  Exception: Dashboard "recent 5 txns" IS cached (see Layer 3).   │    │
│  └──────────────────────────────────────────────────────────────────┘    │
│                                                                          │
│  LAYER 3: REDIS (Specific, high-frequency items)                         │
│  ┌──────────────────────────────────────────────────────────────────┐    │
│  │                                                                  │    │
│  │  ┌─ recent-txns:{accountId} ─── LIST (5 items) ─── TTL: 30s ──┐  │    │
│  │  │  The dashboard's "Recent Transactions" widget.              │  │    │
│  │  │  Updated via Kafka event on every new transaction.          │  │    │
│  │  │  LPUSH + LTRIM to 5 items. Sub-1ms read.                    │  │    │
│  │  └─────────────────────────────────────────────────────────────┘  │    │
│  │                                                                  │    │
│  │  ┌─ txn-count:{accountId}:{month} ─── STRING ─── TTL: 60s ────┐  │    │
│  │  │  Cached transaction count per account per month.            │  │    │
│  │  │  Used to show "234 transactions" without running COUNT(*)   │  │    │
│  │  │  on every page load. Invalidated by Kafka event.            │  │    │
│  │  └─────────────────────────────────────────────────────────────┘  │    │
│  │                                                                  │    │
│  │  ┌─ facets:{accountId}:{dateRange} ─── HASH ─── TTL: 30s ─────┐  │    │
│  │  │  Cached facet counts (byCategory, byStatus, byType).        │  │    │
│  │  │  The base facets (before refinement filters) are cached     │  │    │
│  │  │  because they change only when new transactions arrive.     │  │    │
│  │  └─────────────────────────────────────────────────────────────┘  │    │
│  │                                                                  │    │
```

```
│  │  Full transaction list results are NOT cached in Redis.          │  │
│  │  (Combinatorial explosion — see Caching Strategy doc, Sec 5.3)   │  │
│  └──────────────────────────────────────────────────────────────────┘  │
│                                                                         │
│  LAYER 4: POSTGRESQL (Database-level caching)                           │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │  • shared_buffers (8GB): Hot table pages and index pages are     │  │
│  │    kept in PG's buffer cache. For a user who loads page 1, then  │  │
│  │    page 2 moments later, the data pages are already in buffer.   │  │
│  │                                                                  │  │
│  │  • OS page cache: Linux filesystem cache holds recently-read     │  │
│  │    pages. Effective cache size = 24GB (3x shared_buffers).       │  │
│  │                                                                  │  │
│  │  • Materialized Views: Pre-computed daily/monthly summaries      │  │
│  │    for the dashboard (total credit, total debit, txn count).     │  │
│  │    Refreshed every 15 minutes by the Scheduler Service.          │  │
│  │                                                                  │  │
│  │  • PgBouncer: Connection pool. 300 application connections →     │  │
│  │    50 actual PG connections. Eliminates connection creation      │  │
│  │    overhead (which adds 5-20ms per query without pooling).       │  │
│  └──────────────────────────────────────────────────────────────────┘  │
│                                                                         │
│  LAYER 5: ELASTICSEARCH (Search-specific caching)                       │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │  • Request cache: ES caches entire shard-level query results.    │  │
│  │    If two users search for "Swiggy" with the same filters, the   │  │
│  │    second query is served from ES request cache.                 │  │
│  │    Invalidated automatically on index refresh (~1s intervals).   │  │
│  │                                                                  │  │
│  │  • Filesystem cache: ES relies heavily on OS page cache for      │  │
│  │    Lucene segment files. 50% of node RAM is left for OS cache.   │  │
│  └──────────────────────────────────────────────────────────────────┘  │
└────────────────────────────────────────────────────────────────────────┘
```

5.2 Cache Invalidation for Transaction List

```
New transaction completes
      │
      ▼
Transaction Service publishes Kafka event (txn-events)
      │
      ├──► Cache Invalidation Consumer:
      │      DEL recent-txns:{accountId}        (Redis)
      │      DEL txn-count:{accountId}:{month}  (Redis)
      │      DEL facets:{accountId}:*            (Redis)
      │
      ├──► ES Index Consumer:
      │      Bulk-index new doc to Elasticsearch
      │      (ES request cache auto-invalidated on refresh)
      │
      └──► BFF-Web (Kafka consumer → WebSocket):
             Push event to React client via WebSocket
                   │
                   ▼
             React receives WS event:
             dispatch(transactionApi.util.invalidateTags(['Transactions']))
             → RTK Query refetches current view
             → UI updates seamlessly (new txn appears at top of list)
```

---

6. Database Partitioning — Handling Billions of Rows

6.1 Partitioning Scheme

```sql
CREATE TABLE transactions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id      UUID NOT NULL,
    amount          NUMERIC(18,2) NOT NULL,
    type            VARCHAR(10) NOT NULL,
    category        VARCHAR(20) NOT NULL,
    status          VARCHAR(10) NOT NULL,
    description     TEXT,
    reference_id    VARCHAR(64),
    idempotency_key VARCHAR(64) UNIQUE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);
```

6.2 Why Monthly Partitions?

Granularity	Pros	Cons	Verdict
Daily	Very fine-grained pruning. Small partitions (10M rows).	365 partitions/year → query planner overhead. Too many child tables to manage.	Rejected — too many partitions
Weekly	Moderate pruning. ~75M rows/partition.	52 partitions/year. Still high management overhead. Non-standard date boundaries.	Rejected — unnatural boundary
Monthly (chosen)	12 partitions/year. "Last 30 days" hits 1-2 partitions. "Last 90 days" hits 3-4. Natural date boundary for archival.	Each partition is ~300M rows — large but manageable with proper indexing.	**Chosen**
Quarterly	Only 4 partitions/year. Simple.	"Last 30 days" still scans a 900M-row partition. Pruning benefit is weak.	Rejected — too coarse

6.3 Partition Management (pg_partman)

```
-- Automated partition creation via pg_partman
SELECT partman.create_parent(
    p_parent_table := 'public.transactions',
    p_control := 'created_at',
    p_type := 'range',
    p_interval := '1 month',
    p_premake := 3             -- pre-create 3 months ahead
);

-- Scheduled maintenance (runs via Scheduler Service, daily):
SELECT partman.run_maintenance('public.transactions');
-- Creates upcoming partitions, detaches old ones if configured
```

6.4 How Partition Pruning Saves Performance

```
Query: WHERE account_id = 'ACC-001' AND created_at ≥ '2026-02-15' AND created_at ≤ '2026-04-15'

Without partitioning:
  Full table scan of 3.6 BILLION rows → minutes

With monthly partitions:
  PG query planner prunes:
  ✗ transactions_2025_01 through transactions_2026_01  (skipped entirely)
  ✓ transactions_2026_02  (scanned — contains rows from Feb 15 onward)
  ✓ transactions_2026_03  (scanned — entire month)
  ✓ transactions_2026_04  (scanned — contains rows through Apr 15)
  ✗ transactions_2026_05 onward  (skipped entirely)

  Only 3 of 36+ partitions scanned (~900M rows instead of 3.6B)
  Within each partition, the index narrows further to just ACC-001's rows
  Final rows touched: ~50 (the user's 50 transactions in that period)
```

6.5 Partition Archival

```
┌──────────────────────────────────────────────────────────────────┐
│  PARTITION LIFECYCLE                                              │
│                                                                  │
│  HOT (0-12 months)                                                │
│  • Primary PostgreSQL + Read Replicas                             │
│  • Full indexes active                                            │
│  • Actively queried by users                                      │
│                                                                  │
│  WARM (12-36 months)                                              │
│  • Detached from primary table → separate table/schema            │
│  • Moved to cheaper PG instance (smaller, HDD-backed)             │
│  • Accessed only via "View older transactions" link in UI         │
│  • Queries route to a dedicated "archive" read replica            │
│                                                                  │
│  COLD (36+ months)                                                │
│  • Exported to Parquet format in S3 Glacier                       │
│  • Searchable via Athena (on-demand, seconds-to-minutes latency)  │
│  • Accessed only for compliance/audit requests                    │
│  • PG partition physically deleted after S3 export verified        │
│                                                                  │
│  UI shows: "Transactions older than 3 years are available upon    │
│  request. Contact support or download from Statements section."   │
└──────────────────────────────────────────────────────────────────┘
```

---

7. Indexing Strategy — Making Queries Fast

7.1 Index Map

```sql
-- PRIMARY: Covers the default view (account + date, most recent first)
-- This is the most critical index. Covers 80% of queries.
CREATE INDEX idx_txn_account_date
    ON transactions (account_id, created_at DESC);

-- CATEGORY: For category filter queries
CREATE INDEX idx_txn_category
    ON transactions (category);

-- AMOUNT: For amount range filter queries
CREATE INDEX idx_txn_amount
    ON transactions (amount);

-- STATUS (partial): Only indexes non-SUCCESS rows (which are rare)
-- Full index on status would be 95% SUCCESS values — waste of space
CREATE INDEX idx_txn_status_partial
    ON transactions (status)
    WHERE status ≠ 'SUCCESS';

-- REFERENCE ID: For exact lookups (support/dispute scenarios)
CREATE INDEX idx_txn_reference
    ON transactions (reference_id)
    WHERE reference_id IS NOT NULL;

-- FULL-TEXT SEARCH (fallback when ES is down)
CREATE INDEX idx_txn_fts
    ON transactions USING gin(to_tsvector('english', description));

-- COMPOSITE: For high-frequency filter combination
-- (account + category + date) covers "Show me all UPI txns last month"
CREATE INDEX idx_txn_account_cat_date
    ON transactions (account_id, category, created_at DESC);
```

7.2 Why These Indexes (And Not Others)

Index	Covers Query Pattern	Size Impact	Justification
`(account_id, created_at DESC)`	Default view, date-range queries	~15% of table size	The workhorse. Used by 80% of queries. Leading column on account_id, sorted by date.
`(category)`	Category filter	~5%	Moderate cardinality (7 values). Small index but enables quick category filter.
`(amount)`	Amount range filter	~10%	Numeric range scans (`amount BETWEEN 1000 AND 50000`).
`(status) WHERE ≠ 'SUCCESS'`	Status filter for failed/pending	~0.5%	Partial index — only 5% of rows are non-SUCCESS. Tiny index, very effective.
`(account_id, category, created_at DESC)`	Combined account + category filter	~20%	Covers the second most common pattern. Worth the storage cost.
GIN on tsvector(description)	Text search fallback	~25%	GIN indexes are large but necessary for PostgreSQL full-text search. Only used when ES is down.

7.3 Indexes Per Partition

Each partition inherits indexes from the parent table automatically. PostgreSQL creates per-partition indexes:

```
transactions_2026_04_idx_txn_account_date
transactions_2026_04_idx_txn_category
transactions_2026_04_idx_txn_amount
transactions_2026_04_idx_txn_status_partial
transactions_2026_04_idx_txn_fts
transactions_2026_04_idx_txn_account_cat_date
```

Per-partition indexes are smaller and fit better in memory. A 300M-row partition's `(account_id, created_at)` index is ~15GB — fits entirely in `shared_buffers` for hot partitions.

7.4 Index-Only Scans

For the count query, we can use an index-only scan if the index covers all needed columns:

```sql
-- This query can be satisfied entirely from the index,
-- without touching the table heap (no "random I/O" penalty)
SELECT COUNT(*)
FROM transactions
WHERE account_id = 'ACC-001'
  AND created_at ≥ '2026-03-16'
  AND created_at ≤ '2026-04-15';

-- Uses: idx_txn_account_date (covers both columns)
-- No heap access needed → very fast (~3-5ms)
```

---

8. Query Execution Plan Analysis

8.1 Default View — Last 30 Days, Single Account

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM transactions
WHERE account_id = 'ACC-001'
  AND created_at ≥ '2026-03-16'
  AND created_at ≤ '2026-04-15'
ORDER BY created_at DESC
LIMIT 20 OFFSET 0;
```

```
Limit  (cost=0.56..45.23 rows=20 width=256) (actual time=0.08..0.15 rows=20 loops=1)
  Buffers: shared hit=8
  →  Append  (cost=0.56..312.00 rows=140 width=256) (actual time=0.07..0.14 rows=20 loops=1)
        Subplans Removed: 34  ← partition pruning eliminated 34 of 36 partitions
        →  Index Scan Backward using transactions_2026_04_idx_account_date
              on transactions_2026_04  (cost=0.56..156.00 rows=85 width=256)
              (actual time=0.06..0.10 rows=20 loops=1)
              Index Cond: (account_id = 'ACC-001' AND created_at ≥  ...  AND created_at ≤  ... )
              Buffers: shared hit=5
        →  Index Scan Backward using transactions_2026_03_idx_account_date
              on transactions_2026_03  (never executed — LIMIT satisfied from 2026_04)
Planning Time: 0.8 ms
Execution Time: 0.3 ms
```

Key observations:

- 34 of 36 partitions pruned — only 2 scanned
- LIMIT 20 satisfied from the first partition alone (April has enough rows)
- March partition never even executed — PG stopped after 20 rows
- All buffer reads are `shared hit` — data was in memory (no disk I/O)
- Total: 0.3ms query execution

8.2 Expensive Query — 90 Days, Multi-Account, Category Filter

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM transactions
WHERE account_id IN ('ACC-001', 'ACC-002', 'ACC-003')
  AND created_at ≥ '2026-01-15'
  AND created_at ≤ '2026-04-15'
  AND category IN ('UPI', 'NEFT')
  AND status = 'SUCCESS'
ORDER BY created_at DESC
LIMIT 20 OFFSET 0;
```

```
Limit  (cost=234.56..234.61 rows=20 width=256) (actual time=8.5..8.6 rows=20 loops=1)
  Buffers: shared hit=120, read=15
  →  Sort  (cost=234.56..235.00 rows=180 width=256) (actual time=8.4..8.5 rows=20 loops=1)
        Sort Key: created_at DESC
        Sort Method: top-N heapsort  Memory: 28kB
        →  Append  (cost=0.56..220.00 rows=180 width=256) (actual time=0.1..7.8 rows=180 loops=1)
              Subplans Removed: 32  ← only 4 partitions scanned
              →  Index Scan using transactions_2026_04_idx_account_cat_date
                    on transactions_2026_04
                    Index Cond: (account_id IN (...) AND category IN ('UPI','NEFT'))
                    Filter: (status = 'SUCCESS')
                    Rows Removed by Filter: 5
                    Buffers: shared hit=35, read=4
              →  Index Scan using transactions_2026_03_idx_account_cat_date  ...
              →  Index Scan using transactions_2026_02_idx_account_cat_date  ...
              →  Index Scan using transactions_2026_01_idx_account_cat_date  ...
Planning Time: 1.2 ms
Execution Time: 9.1 ms
```

Key observations:

- 4 partitions scanned (Jan–Apr) — the rest pruned
- Composite index `(account_id, category, created_at)` covers the main filter
- Status filter applied as post-filter (partial index only covers non-SUCCESS)
- Some disk reads (`read=15`) — older partitions not fully in buffer cache
- Top-N heapsort (28KB memory) — PG only sorts enough to find the top 20
- Total: 9.1ms

---

9. UI Performance — Loading States, Skeletons & Perceived Speed

9.1 The Loading State Machine

```
┌──────────────────────────────────────────────────────────────────────┐
│              TRANSACTION LIST — UI STATE MACHINE                     │
│                                                                      │
│  ┌──────────────┐      cache hit      ┌──────────────┐              │
│  │   INITIAL    │ ──────────────────→ │    DATA      │              │
│  │  (mount)     │                     │  (rendered)  │              │
│  └──────────────┘                     └──────────────┘              │
│         │ cache miss                        │ filter change / refetch │
│         ▼                                   ▼                        │
│  ┌──────────────┐      data arrives   ┌──────────────┐              │
│  │  SKELETON    │ ──────────────────→ │    DATA      │              │
│  │ (first load) │                     │  (updated)   │              │
│  └──────────────┘                     └──────────────┘              │
│         │ > 5 seconds                       │ stale refetch           │
│         ▼                                   ▼                        │
│  ┌──────────────┐                     ┌──────────────┐              │
│  │  SLOW LOAD   │                     │ STALE +      │  ← show old data,
│  │ (show msg)   │                     │ REFETCHING   │    subtle spinner in
│  └──────────────┘                     └──────────────┘    corner     │
│         │ > 15 seconds                                               │
│         ▼                                                            │
│  ┌──────────────┐      retry          ┌──────────────┐              │
│  │  TIMEOUT     │ ──────────────────→ │  SKELETON    │              │
│  │ (error UI)   │                     │ (retrying)   │              │
│  └──────────────┘                     └──────────────┘              │
│         │ 3 retries failed                                           │
│         ▼                                                            │
│  ┌──────────────┐                                                    │
│  │   ERROR      │                                                    │
│  │ (full error  │                                                    │
│  │  screen)     │                                                    │
│  └──────────────┘                                                    │
└──────────────────────────────────────────────────────────────────────┘
```

9.2 Skeleton Screen (First Load / Cache Miss)

When there's no cached data, show a skeleton immediately (0ms):

```
┌──────────────────────────────────────────────────────────────────────┐
│  Transactions         [Savings ▼] [Last 30 days ▼] [All categories ▼]│
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │  ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   ▒▒▒▒▒▒▒▒          ▒▒▒▒▒▒▒▒              │   │
│   │  ▒▒▒▒▒▒▒▒▒▒▒▒       Apr 14, 2026       − ₹ ▒▒▒▒            │   │
│   ├──────────────────────────────────────────────────────────────┤   │
│   │  ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   ▒▒▒▒▒▒▒▒          ▒▒▒▒▒▒▒▒              │   │
│   │  ▒▒▒▒▒▒▒▒▒▒▒▒       Apr 14, 2026       + ₹ ▒▒▒▒            │   │
│   ├──────────────────────────────────────────────────────────────┤   │
│   │  ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   ▒▒▒▒▒▒▒▒          ▒▒▒▒▒▒▒▒              │   │
│   │  ▒▒▒▒▒▒▒▒▒▒▒▒       Apr 13, 2026       − ₹ ▒▒▒▒            │   │
│   ├──────────────────────────────────────────────────────────────┤   │
│   │  ( ... 7 more skeleton rows, shimmer animation ... )         │   │
│   └──────────────────────────────────────────────────────────────┘   │
│                                                                      │
│  ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  (pagination skeleton)                         │
└──────────────────────────────────────────────────────────────────────┘

Skeleton shows:
• 10 rows (matching page size roughly)
• Shimmer animation (CSS animation, no JS — zero jank)
• Filter panel is ALREADY INTERACTIVE (user can start selecting)
• Layout matches the real data exactly (no layout shift when data arrives)
```

9.3 Implementation

```
function TransactionList() {
  const { filters, dispatch } = useTransactionFilters();
  const { data, isLoading, isFetching, isError, error } = useGetTransactionsQuery(
    filters,
    { pollingInterval: 0, refetchOnMountOrArgChange: 30 }
  );

  return (
    <div>
      {/* Filter panel — always interactive, even during loading */}
      <FilterPanel filters={filters} dispatch={dispatch} />
      <FilterChips filters={filters} dispatch={dispatch} />

      {/* Fetching indicator (stale-while-revalidate mode) */}
      {isFetching && !isLoading && (
        <div className="h-1 w-full">
          <div className="h-full bg-blue-500 animate-progress-bar" />
        </div>
      )}

      {/* State machine rendering */}
      {isLoading ? (
        <TransactionSkeleton rows={10} />
      ) : isError ? (
        <TransactionError error={error} onRetry={() => dispatch({ type: 'RESET_ALL' })} />
      ) : data?.results.length === 0 ? (
        <EmptyState filters={filters} onClear={() => dispatch({ type: 'RESET_ALL' })} />
      ) : (
        <>
          <p className="text-sm text-gray-500">
            Showing {data.results.length} of {data.pagination.totalElements} transactions
          </p>
          <TransactionTable
            transactions={data.results}
            highlights={data.highlights}
          />
          <Pagination
            page={filters.page}
            totalPages={data.pagination.totalPages}
            onChange={(p) => dispatch({ type: 'SET_PAGE', payload: p })}
          />
        </>
      )}
    </div>
  );
}
```

9.4 Stale-While-Revalidate (No Loading Spinner on Refetch)

When the user already has data visible but it's stale (30s+ old) and a refetch is triggered:

```
┌──────────────────────────────────────────────────────────┐
│  ═══════════════════════════════  (thin progress bar)     │
│                                                          │
│  Transactions  [Savings ▼] [Last 30 days ▼]              │
│                                                          │
│   ┌──────────────────────────────────────────────────┐   │
│   │  Paid to Swiggy - Order #1234                    │   │
│   │  UPI • Apr 14, 2026                  - ₹450      │   │
│   ├──────────────────────────────────────────────────┤   │
│   │  Salary Credit                                   │   │
│   │  NEFT • Apr 14, 2026               + ₹85,000     │   │
│   ├──────────────────────────────────────────────────┤   │
│   │  ( ...  stale data still shown, fully interactive) │   │
│   └──────────────────────────────────────────────────┘   │
│                                                          │
│  User sees: old data + thin progress bar at top           │
│  User does NOT see: loading spinner, skeleton, blank page │
│                                                          │
│  When fresh data arrives:                                 │
│  → Table smoothly updates (rows may reorder/change)       │
│  → Progress bar disappears                                │
│  → If new transaction appeared, it slides in at the top   │
└──────────────────────────────────────────────────────────┘
```

9.5 Empty State

When filters return zero results:

```
┌──────────────────────────────────────────────────────────┐
│  Active filters: [UPI ×] [> ₹1 Lakh ×] [Failed ×]      │
│                                                          │
│              ┌────────────────────┐                      │
│              │                    │                      │
│              │   (illustration)   │                      │
│              │   No transactions  │                      │
│              │   found            │                      │
│              │                    │                      │
│              └────────────────────┘                      │
│                                                          │
│  No transactions match your current filters.              │
│                                                          │
│  Try:                                                    │
│  • Expanding the date range                              │
│  • Removing some filters  [Clear all filters]            │
│  • Checking a different account                          │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

---

10. When Data Takes Too Long — Timeout UX Patterns

10.1 Timeout Tiers

Duration	State	UI Action
0–300ms	Normal	Skeleton is showing. User perceives instant start.
300ms–2s	Acceptable	Skeleton continues. No additional messaging. User is patient.
2s–5s	Slow	Skeleton continues. Show subtle text: "Loading your transactions..."
5s–10s	Very slow	Show: "This is taking longer than usual. Still loading..." with a [Cancel] button.
10s–15s	Timeout warning	Show: "Your request is taking too long. We'll retry automatically." Auto-retry with narrower date range (optimization).
15s+	Failed	Show error screen with retry button and troubleshooting suggestions. Log to monitoring.

10.2 Progressive Timeout UI

```
function useLoadingMessage(isLoading: boolean) {
  const [message, setMessage] = useState<string | null>(null);
  const [showCancel, setShowCancel] = useState(false);
  const startTime = useRef<number>(0);

  useEffect(() => {
    if (!isLoading) {
      setMessage(null);
      setShowCancel(false);
      return;
    }

    startTime.current = Date.now();

    const timer2s = setTimeout(() => setMessage('Loading your transactions...'), 2000);
    const timer5s = setTimeout(() => {
      setMessage('This is taking longer than usual...');
      setShowCancel(true);
    }, 5000);
    const timer10s = setTimeout(() =>
      setMessage('Still working on it. The server may be busy.'), 10000);

    return () => {
      clearTimeout(timer2s);
      clearTimeout(timer5s);
      clearTimeout(timer10s);
    };
  }, [isLoading]);

  return { message, showCancel };
}
```

10.3 Auto-Retry with Narrowed Scope

If the original request times out (15s), the system automatically retries with a smaller date range:

```
const { data, isError, error } = useGetTransactionsQuery(filters, {
  // RTK Query retry configuration
  extraOptions: {
    maxRetries: 2,
    backoff: (attempt) => Math.min(1000 * 2 ** attempt, 5000),
  },
});

useEffect(() => {
  if (isError && error?.status === 504) {
    // Gateway timeout - retry with narrower date range
    const narrowedRange = {
      from: thirtyDaysAgo(),  // original might have been 90 days
      to: today(),
    };
    dispatch({ type: 'SET_DATE_RANGE', payload: narrowedRange });
    showToast('Showing last 30 days for faster results. You can expand the range.');
  }
}, [isError]);
```

10.4 Error Screen (All Retries Failed)

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│              ┌────────────────────┐                      │
│              │                    │                      │
│              │   (error icon)     │                      │
│              │                    │                      │
│              └────────────────────┘                      │
│                                                          │
│  Unable to load transactions                              │
│                                                          │
│  This could be a temporary issue. Here's what you can    │
│  try:                                                    │
│                                                          │
│  1. Check your internet connection                       │
│  2. Narrow the date range to "Last 7 days"               │
│  3. Select a single account                              │
│                                                          │
│  [ Try Again ]     [ Load Last 7 Days ]                  │
│                                                          │
│  If this keeps happening, contact support.               │
│  Error reference: ERR-TXN-504-abc123                     │
└──────────────────────────────────────────────────────────┘
```

11. When It's Consistently Slow — Root Cause Diagnosis & Fixes

11.1 Diagnostic Flowchart

```
Transaction list is consistently slow (p99 > 500ms)
      │
      ▼
┌── Check: Where is the time spent? ──────────────────────────┐
│                                                              │
│  Step 1: Jaeger distributed trace for a slow request        │
│          Look at span durations:                             │
│                                                              │
│  BFF ──────► Txn Service ──────► PostgreSQL                  │
│  5ms         3ms                 ???ms                       │
│                                                              │
│  If PG span is dominant → database issue (go to Step 2)     │
│  If Network spans are dominant → infrastructure issue (Step 5)  │
│  If BFF span is dominant → BFF issue (Step 6)               │
└──────────────────────────────────────────────────────────────┘
      │
      ▼ (PG is slow)
┌── Step 2: Check EXPLAIN ANALYZE for the query ──────────────┐
│                                                              │
│  Run: EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) SELECT ...     │
│                                                              │
│  Look for:                                                   │
│  ┌──────────────────────────────────────────────────────┐   │
│  │ SYMPTOM                      | DIAGNOSIS                 │   │
│  │──────────────────────────────┼──────────────────────────│   │
│  │ "Seq Scan" on transactions   | Missing index or partition │   │
│  │                              | pruning not working       │   │
│  │                              |                           │   │
│  │ "Buffers: read=50000"        | Data not in memory. Cold  │   │
│  │ (high disk reads)            | partition. Increase       │   │
│  │                              | shared_buffers or warm cache │   │
│  │                              |                           │   │
│  │ "Sort Method: external       | Sort spilling to disk. query │   │
│  │  merge"                      | returns too many rows before │   │
│  │                              | LIMIT. Need better index. │   │
│  │                              |                           │   │
│  │ "Rows Removed by Filter:     | Index not selective enough. │   │
│  │  50000"                      | Too many rows fetched,    │   │
│  │                              | then filtered. Add composite │   │
│  │                              | index matching filter combo. │   │
│  │                              |                           │   │
│  │ "Subplans Removed: 0"        | Partition pruning NOT working! │   │
│  │                              | Check: is created_at in WHERE? │   │
│  │                              | Check: enable_partition_pruning │   │
│  └──────────────────────────────┴──────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
      │
      ▼
┌── Step 3: Check for lock contention ────────────────────────┐
│                                                              │
│  SELECT * FROM pg_stat_activity WHERE wait_event_type = 'Lock';  │
│                                                              │
│  If read queries are waiting on write locks:                 │
```

```
│  → Route reads to read replica (@Transactional(readOnly = true))   │
│  → Read replicas don't contend with write primary                  │
└────────────────────────────────────────────────────────────────────┘
      │
      ▼
┌── Step 4: Check for bloat / VACUUM ────────────────────────────────┐
│                                                                    │
│  SELECT schemaname, relname, n_dead_tup, last_autovacuum           │
│  FROM pg_stat_user_tables WHERE relname LIKE 'transactions_%';     │
│                                                                    │
│  If n_dead_tup is very high and last_autovacuum was long ago:      │
│  → Table bloat. Indexes scan dead tuples. Run VACUUM ANALYZE.      │
│  → Tune: autovacuum_vacuum_scale_factor = 0.01 (default 0.2 is    │
│    too aggressive for large tables — VACUUM triggers too late)     │
└────────────────────────────────────────────────────────────────────┘
      │
      ▼
┌── Step 5: Check connection pool ───────────────────────────────────┐
│                                                                    │
│  PgBouncer stats: SHOW POOLS;                                      │
│                                                                    │
│  If cl_waiting > 0: requests waiting for a DB connection           │
│  → Increase PgBouncer pool size or add read replica                │
│  → Check for long-running transactions holding connections         │
└────────────────────────────────────────────────────────────────────┘
      │
      ▼
┌── Step 6: Check network / infrastructure ──────────────────────────┐
│                                                                    │
│  If Jaeger shows network spans > 20ms between services:            │
│  → Cross-AZ traffic? Ensure BFF and Txn Service in same AZ.       │
│  → DNS resolution? Check if service discovery is slow.             │
│  → gRPC channel exhaustion? Check connection pool settings.        │
└────────────────────────────────────────────────────────────────────┘
```

11.2 Specific Fix Playbook

Problem	Symptom	Fix	Time to Effect
Missing date filter	Full table scan across all partitions	Enforce date range in API validation. Default to "last 30 days" if not specified.	Immediate (code change)
Deep pagination	OFFSET > 10,000, slow query	Block deep pagination. Switch to cursor-based.	Immediate
Missing composite index	Seq scan on category + status	`CREATE INDEX CONCURRENTLY idx_txn_account_cat_date ON transactions (account_id, category, created_at DESC)`	Minutes (concurrent, no downtime)
Cold partition (not in cache)	High `Buffers: read` count	Run a warm-up query after partition creation: `SELECT count(*) FROM transactions_2026_04`	Minutes
Connection pool exhaustion	Requests queuing at PgBouncer	Increase pool size OR add read replica OR optimize slow queries holding connections	Minutes to hours
Table bloat	n_dead_tup > 10M on a partition	`VACUUM (VERBOSE, ANALYZE) transactions_2026_04`	Minutes
Stale statistics	PG query planner makes bad choices	`ANALYZE transactions_2026_04` (updates row count estimates)	Seconds
Read-write contention	Read queries blocked by write locks	Route reads to read replica. Already architected but verify routing is working.	Immediate (config)
Too many columns returned	500-byte rows × 20 rows × many concurrent users	Use DTO projection: `SELECT id, amount, type, category, status, description, created_at` instead of `SELECT *`	Code change
Count query is slow	`COUNT(*)` on large result set	Use estimated count for > 10K results. Cache exact count in Redis.	Code change

---

12. Read Replica & Connection Pool Strategy

12.1 Read Routing

```
┌──────────────────────────────────────────────────────────────────┐
│                  READ/WRITE ROUTING                               │
│                                                                  │
│  Transaction Service                                             │
│       │                                                          │
│       ├── @Transactional                                         │
│       │   (readOnly = false)  ──────► PG PRIMARY                 │
│       │   Used by: createTransaction(), debit(), credit()        │
│       │                                                          │
│       └── @Transactional                                         │
│           (readOnly = true)   ──────► PG READ REPLICA            │
│           Used by: list(), search(), count(), export()           │
│                                                                  │
│  Routing via AbstractRoutingDataSource:                          │
│  if TransactionSynchronizationManager.isCurrentTransactionReadOnly()│
│    → route to "replica" DataSource                               │
│  else                                                            │
│    → route to "primary" DataSource                               │
└──────────────────────────────────────────────────────────────────┘
```

12.2 Connection Pool Sizing

```
┌──────────────────────────────────────────────────────────┐
│  PgBouncer Configuration                                  │
│                                                           │
│  [databases]                                              │
│  txn_db_primary = host=pg-primary port=5432 dbname=txn_db │
│  txn_db_replica = host=pg-replica-1 port=5432 dbname=txn_db│
│                                                           │
│  [pgbouncer]                                              │
│  pool_mode = transaction                                  │
│  max_client_conn = 500                                    │
│  default_pool_size = 30   (primary)                       │
│  reserve_pool_size = 5                                    │
│  reserve_pool_timeout = 3                                 │
│                                                           │
│  Replica pool:                                            │
│  default_pool_size = 50   (more connections for reads)    │
│                                                           │
│  Rationale:                                               │
│  • 20 Txn Service pods × 10 connections each = 200 app conns│
│  • PgBouncer multiplexes 200 → 30 PG connections (primary)│
│  • PgBouncer multiplexes 200 → 50 PG connections (replica)│
│  • Connection creation overhead avoided (10-20ms savings)  │
└──────────────────────────────────────────────────────────┘
```

13. Materialized Views & Pre-Computed Summaries

13.1 Dashboard Summary View

The dashboard doesn't need raw transaction rows — it needs aggregates. Pre-compute them:

```sql
CREATE MATERIALIZED VIEW mv_daily_txn_summary AS
SELECT
    account_id,
    DATE(created_at) AS txn_date,
    COUNT(*)         AS txn_count,
    SUM(CASE WHEN type = 'CREDIT' THEN amount ELSE 0 END) AS total_credit,
    SUM(CASE WHEN type = 'DEBIT'  THEN amount ELSE 0 END) AS total_debit,
    COUNT(CASE WHEN category = 'UPI'  THEN 1 END) AS upi_count,
    COUNT(CASE WHEN category = 'NEFT' THEN 1 END) AS neft_count,
    COUNT(CASE WHEN category = 'RTGS' THEN 1 END) AS rtgs_count,
    COUNT(CASE WHEN status = 'FAILED' THEN 1 END) AS failed_count
FROM transactions
WHERE created_at ≥ CURRENT_DATE - INTERVAL '90 days'
GROUP BY account_id, DATE(created_at);

CREATE UNIQUE INDEX idx_mv_daily ON mv_daily_txn_summary (account_id, txn_date);

-- Refreshed every 15 minutes by the Scheduler Service
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_txn_summary;
```

Dashboard query hits the materialized view instead of the raw transaction table:

```sql
-- "Show me this month's summary" — instant, no scanning 300M rows
SELECT SUM(txn_count), SUM(total_credit), SUM(total_debit)
FROM mv_daily_txn_summary
WHERE account_id = 'ACC-001'
  AND txn_date ≥ '2026-04-01';
-- Result: 0.5ms (the MV has ~90 rows per account for 90 days)
```

14. Performance Budget — Latency Breakdown Per Layer

14.1 Latency Budget Allocation

```
Total allowed: 300ms (p99) for transaction list load

┌──────────────────────────────────────────────────────────────┐
│  Layer                    | Budget (p99)  | Current (p99)    |
│  ─────────────────────────┼───────────────┼──────────────────┤
│  React (dispatch + render)|    15ms       |    10ms          |
│  Network (round-trip)     |    60ms       |    30ms          |
│  API Gateway              |    15ms       |    10ms          |
│  BFF (validate + route)   |    15ms       |     8ms          |
│  Txn Service (build spec) |    10ms       |     5ms          |
│  PG Query (data)          |    80ms       |    15ms          |
│  PG Query (count)         |    40ms       |     8ms          |
│  DTO Mapping              |    10ms       |     5ms          |
│  Response serialization   |    10ms       |     5ms          |
│  Buffer/headroom          |    45ms       |     —            |
│  ─────────────────────────┼───────────────┼──────────────────┤
│  TOTAL                    |   300ms       |    96ms          |
│                                                              |
│  Current utilization: 32% of budget (68% headroom)           |
│  When PG queries degrade, we have 200ms+ of headroom.        |
└──────────────────────────────────────────────────────────────┘
```

14.2 SLO Definition

Metric	SLO	Measurement
Transaction list load (p50)	< 100ms	Prometheus histogram bucket
Transaction list load (p95)	< 200ms	Prometheus histogram bucket
Transaction list load (p99)	< 500ms	Prometheus histogram bucket
First Contentful Paint (FCP)	< 500ms	Real User Monitoring (RUM)
Largest Contentful Paint (LCP)	< 1.5s	RUM
Time to Interactive (TTI)	< 2.0s	RUM
Skeleton to data (perceived load)	< 1.0s	RUM (custom metric)

15. Load Testing & Performance Validation

15.1 Gatling Test Scenarios

Scenario	Users	Duration	Query Pattern	Target
Normal load	1,000 concurrent	30 min	80% page 1, 15% page 2-5, 5% filter changes	p99 < 300ms
Peak load	10,000 concurrent	15 min	Same as normal with 3x filter changes	p99 < 500ms
Deep pagination stress	500 concurrent	10 min	Random pages 1-500 per user	p99 < 1s (with guard at page 500)
Cold start	5,000 concurrent	5 min	First load (no cache), random date ranges	p99 < 800ms
Search + filters	2,000 concurrent	20 min	30% text search + 70% filter-only	p99 < 500ms

15.2 Performance Regression Gate

Every pull request runs a performance test in staging:

```
CI Pipeline:
1. Deploy to staging
2. Run Gatling "normal load" scenario (1,000 users, 5 min)
3. Compare p99 latency against main branch baseline
4. If p99 degrades by > 20%:
   → Block merge
   → Alert: "Performance regression detected. p99 went from 120ms to 180ms."
5. If p99 improves by > 10%:
   → Comment: "Performance improvement detected!"
```

16. Performance Monitoring & Alerting

16.1 Key Metrics (Prometheus)

Metric	Labels	Alert
`txn_list_latency_seconds`	`{endpoint, method, status}`	p99 > 500ms for 5 min
`txn_list_db_query_seconds`	`{query_type: data|count}`	p99 > 200ms for 5 min
`txn_list_cache_hit_ratio`	`{cache: rtk|redis}`	< 80% for 10 min
`txn_list_error_rate`	`{status: 5xx|timeout}`	> 1% for 2 min
`pg_replication_lag_seconds`	`{replica}`	> 5s for 2 min
`pgbouncer_waiting_clients`	`{pool}`	> 0 for 1 min
`txn_list_pagination_depth`	`{type: offset|cursor}`	offset > 5000 (anomaly)
`txn_list_slow_query_count`	`{threshold: 500ms}`	> 10 per minute

16.2 Grafana Dashboard

```
┌──────────────────────────────────────────────────────────────────────┐
│  TRANSACTION LIST PERFORMANCE DASHBOARD                              │
│                                                                      │
│  ┌──────────────────────┐  ┌──────────────────────┐  ┌────────────┐│
│  │ Latency (p50/p95/p99│  │ Request Rate         │  │ Error Rate │
│  │      85 / 130 / 210 │  │     3,200 req/sec    │  │   0.02%    │
│  │  ▓▓▓
```